

PilotTrace

PilotTrace is an illustrative, offline Python prototype for validating and scoring caller-mapped numeric trace data. It accepts UTF-8 JSON inputs; it does not establish compatibility with external equipment or actual telemetry.

Each accepted frame has exactly 512 finite numeric values. The program retains valid frames, reports every rejected frame with its zero-based index and reasons, divides each accepted frame into 16 contiguous blocks of 32 values, and scores each block:

Plain Text

```
score = 0.45 * max(abs(x)) + 0.35 * RMS(x) + 0.20 * mean(abs(x))
```

A block is flagged when `score >= threshold`. The finite, non-negative threshold is required in the manifest, so it is frozen for that run and covered by the manifest SHA-256. PilotTrace does not calibrate or tune it. A frame is flagged if any block is flagged.

Boundaries

This tool processes offline numeric data that a caller has already mapped. It applies no normalization, unit conversion, baseline correction, or other preprocessing. It makes no external connections and is not an alarm, safety, medical, or process-control system. A replay can demonstrate input compatibility and deterministic arithmetic; it cannot establish field or detection performance.

When boolean labels are supplied, the report computes explicitly named descriptive frame-classification metrics using accepted labeled frames only. A frame is predicted positive when at least one block is flagged. Unlabeled frames do not enter metric denominators. When no accepted frame has a label, no classification metrics are computed; the report gives a status and reason instead. These descriptive figures are not estimates of performance on other data.

Run the package CLI

Python 3.11 or later; runtime and tests use only the Python standard library. From this directory:

Bash

```
python3 tools/make_example.py
python3 -m pilottrace \
  --trace examples/sample_trace.json \
```

```
--manifest examples/sample_manifest.json \  
--output examples/sample_report.json  
python3 -m unittest discover -s tests -v
```

Omit `--output` or pass `--output -` to write the report to standard output. Invalid JSON, a malformed document root, an invalid manifest or threshold, or file errors produce an error on standard error and a non-zero exit. If the trace document is readable but individual frames are invalid, the CLI writes a report with accepted results and every rejected frame; one rejected frame does not prevent processing the others.

The report is deterministic for identical trace and manifest bytes. It includes SHA-256 hashes of the exact input bytes, scoring version, threshold, all 16 scores per accepted frame, rejected-frame reasons, flagged blocks, and descriptive label metrics only when labels are present. It has no generation timestamp or platform-dependent data.

Standalone file

`pilottrace_standalone.py` contains the validation, scoring, reporting, file I/O, and command-line runtime in one standard-library-only Python file. Copy that file by itself when a self-contained command is useful; it does not import the `pilottrace` package. The standalone command uses the same sample inputs:

Bash

```
python3 pilottrace_standalone.py \  
--trace examples/sample_trace.json \  
--manifest examples/sample_manifest.json \  
--output /tmp/pilottrace-report.json
```

The standalone file processes caller-mapped numeric data only. It is **not validated operational detection** and does not provide alerts or control functions. Its calculations demonstrate schema validation and deterministic scoring, not validated capability.

Python API

Python

```
from pilottrace import analyze_files, report_json  
  
report = analyze_files("examples/sample_trace.json", "examples/sample_manifest.  
print(report_json(report))
```

The lower-level `analyze_bytes(trace_bytes, manifest_bytes)` API is also available from `pilottrace` for callers that already hold the exact input bytes. Public helpers `score_block(values)` and `score_frame(values)` are exported for direct deterministic scoring.

JSON schemas

Trace document

The top level is exactly one object with a non-empty `frames` array:

JSON

```
{
  "frames": [
    {"values": ["exactly 512 JSON numbers go here"], "label": true},
    {"values": ["exactly 512 JSON numbers go here"]}
  ]
}
```

The quoted phrase is explanatory, not a numeric value; use the generated example for complete valid JSON. Each frame must be an object with required `values` and optional `label` fields, and no other fields. `values` must be an array of exactly 512 finite JSON numbers. Booleans are not numeric values; NaN and positive or negative infinity are rejected. A present `label` must be a JSON boolean (`true` means positive, `false` means negative). Labels may be omitted per frame. Rejected frames are excluded from scoring and metrics.

Manifest

A manifest is exactly one JSON object with `schema_version: 1`, a finite non-negative numeric `threshold`, and all eight non-empty, explicit string declarations below. Placeholder values such as `unknown`, `unspecified`, `N/A`, or `TBD` are rejected. The declarations are recorded in the report and describe the caller's mapping and choices; the program does not verify them against an external source. To change the threshold, create and preserve a correspondingly updated manifest; there is no command-line override or tuning.

JSON

```
{
  "schema_version": 1,
  "threshold": 0.5,
  "channel_order": "Caller-declared index order from 0 through 511, grouped into",
  "units": "dimensionless synthetic amplitude units",
  "scaling": "Values are used as supplied; no additional scaling is applied.",
}
```

```

    "timebase": "Frame order only; no elapsed-time rate is asserted.",
    "baseline_preprocessing": "Zero is the illustrative baseline; no normalization",
    "missing_value_policy": "A frame with a missing, non-numeric, NaN, or infinite",
    "source_provenance": "Deterministic local example data generated by tools/make",
    "permitted_use": "Offline software and schema demonstration only; not operational",
  }

```

For caller-provided data, replace each illustrative declaration with the channel order and block assignment, units, scale, timebase, preprocessing, missing-value policy, provenance, and permitted use that actually apply. The example wording is not a declaration for other data.

Report interpretation

`hashes.trace_input_sha256` and `hashes.manifest_sha256` cover the exact source bytes.

`rejected_frames` gives each rejected zero-based frame index and one or more reasons.

`frames` contains accepted frames with their label (or `null`), frame-level flag, and sixteen block scores and flags. `flagged_blocks` is a compact list of flagged blocks with frame index, block index, score, and half-open value range.

`labeled_frame_classification.status` is `computed_descriptive_only` when at least one accepted frame is labeled, and `not_computed` otherwise. Metrics with a zero denominator are `null`. The prediction is frame-level (one or more flagged blocks); block localization is not computed because the schema defines only frame-level boolean labels.